LAB 5

STATIC CODE ANALYSIS

Bank Account Management Program

STUDENT DETAILS

Name: Saksham Bajpayi

SRN: PES1UG24CS404

Programme: B.Tech - Computer Science and Engineering

Experiment: Lab 5 - Static Code Analysis

Assigned repository: SETAPESU26/L5_40_bank_account_systen

Date: 30 October 2026

Department of Computer Science and Engineering

1. Objective

The objective of this lab was to improve the quality, security and maintainability of a Python program by reviewing it with static analysis tools. The assigned handout introduces Pylint for code quality, Bandit for common security issues and Flake8 for style and formatting. The task also requires documenting findings, correcting issues and checking the revised program.

2. Program Overview

The assigned repository contains a small bank account management program in bank_account_system.py. The original script stores account information in a dictionary and provides functions to create accounts, deposit and withdraw money, calculate a bonus, hash a PIN, generate a temporary PIN and write a backup file. A demonstration is executed at the end of the original file.

The review focused on the original source in the assigned repository. The revised version keeps the same basic purpose while adding input checks, safer handling of expressions and PINs, more reliable file handling and a proper main guard.

3. Tools and Method

The lab handout specifies three tools:

- Pylint: checks code quality, naming, unused elements and likely programming mistakes.
- Bandit: scans Python source for common security problems and unsafe coding patterns.
- Flake8: checks Python formatting and style, including common PEP 8 violations.

The original file was inspected first. The identified code patterns were then corrected in the working copy. After the changes, the Python compiler and a focused set of functional checks were run against the revised program.

4. Issues Identified and Corrections

1. Mutable default argument

Original issue: The createAccount function used an empty list as its default parameter. Python would reuse that same list across calls when no list was supplied.

Correction: Changed the default to None and create a new list inside the function when needed.

Why it matters: Each account creation call now gets independent transaction storage.

2. Overly broad exception handling

Original issue: The deposit function used a bare except block and silently ignored every error.

Correction: Removed the broad exception handler. The function now checks whether the account exists and whether the deposit amount is positive, then returns True or False.

Why it matters: Expected invalid inputs are handled directly, while unrelated programming errors are not hidden.

3. Unsafe eval usage

Original issue: The compute_bonus function passed an expression directly to eval(), which can execute arbitrary Python code.

Correction: Replaced eval() with ast parsing and a restricted evaluator. It accepts the account variable bal, numeric constants and a limited set of arithmetic operators.

Why it matters: Only the intended calculation syntax is accepted. Other expression types raise an error.

4. Weak MD5 PIN hashing

Original issue: The original get_hash function used MD5, which is not suitable for storing PIN or password verifiers.

Correction: Replaced MD5 with PBKDF2-HMAC-SHA256 using a random salt and 200,000 iterations. Added a verification function that compares hashes with hmac.compare_digest().

Why it matters: The stored verifier is salted and computationally more expensive to guess than a plain MD5 digest.

5. Non-cryptographic PIN generation

Original issue: The original temporary PIN generator used random.randint(), which is not intended for security-sensitive values.

Correction: Changed PIN generation to secrets.randbelow() and kept the result within the four-digit range.

Why it matters: The secrets module is designed for generating security-sensitive random values.

6. File handling and shell command construction

Original issue: The original backup function manually opened and closed a file, then passed the filename into os.system().

Correction: Used a with statement to write the backup with UTF-8 encoding. Removed the shell command and moved the fixed completion message into the demonstration function.

Why it matters: The file is closed reliably, and user-controlled filename text is no longer interpreted by a shell.

7. Hard-coded administrator token

Original issue: The original main function contained a fixed administrator token and printed it.

Correction: Removed the unused token and the statement that displayed it.

Why it matters: Secrets should not be embedded in source code or exposed in program output.

8. Unused import and variable

Original issue: The original file imported sys without using it and assigned a service_fee value that did not affect the withdrawal operation.

Correction: Removed the unused import and variable.

Why it matters: This reduces clutter and avoids misleading code that appears to have an effect when it does not.

9. Naming and formatting consistency

Original issue: The original createAccount name used mixed casing and the transaction message used percent-style formatting.

Correction: Renamed the function to create_account, adopted f-strings and formatted the revised code consistently.

Why it matters: Consistent naming and formatting make the program easier to read and maintain.

10. Module executed on import

Original issue: The original script called main() unconditionally at the end of the file.

Correction: Placed the call inside if __name__ == "__main__":.

Why it matters: The module can now be imported for testing without automatically running the demonstration.

5. Verification and Results

The revised source was compiled with Python using py_compile. Compilation completed successfully. A separate functional test run also completed successfully, with all 16 checks passing.

The checks covered account creation, duplicate-account and negative-deposit rejection, independent transaction lists, valid and invalid deposits and withdrawals, supported bonus calculations, rejection of an attempted import and system call, correct and incorrect PIN verification, distinct salts for repeated hashes, four-digit temporary PIN generation and backup-file creation.

The exact result of the local checks was 16 out of 16 passed. This confirms the tested behaviours in the revised program, but it is not a substitute for the three static analysis reports required by the lab.

6. Static Analysis Tool Status

Pylint, Bandit and Flake8 were not run in this workspace because the tools were not installed. An attempt to install them with pip could not reach the package index due to a network name-resolution failure. Therefore, no before-and-after tool reports or Pylint score are claimed in this report.

The following commands can be used in the local repository after installing the tools. Keep a copy of the original source as bank_account_system_original.py, then run the tools on both the original and revised files:

```
# Before: original source
pylint bank_account_system_original.py > pylint_report_before.txt
bandit -r bank_account_system_original.py > bandit_report_before.txt
flake8 bank_account_system_original.py > flake8_report_before.txt

# After: revised source
pylint bank_account_system.py > pylint_report_after.txt
bandit -r bank_account_system.py > bandit_report_after.txt
flake8 bank_account_system.py > flake8_report_after.txt
```

The same commands should be run on the original copy to produce the three before reports, and on the revised copy to produce the three after reports. The final Pylint score and any remaining findings should be recorded from the actual tool output. The three requested repository commits have also not been created or pushed; the GitHub connection available during preparation had read-only permissions.

7. Reflection

This exercise showed that static analysis can identify problems that may not be obvious during a normal run of a program. The most serious issue in this script was the use of eval(), because it allowed a calculation field to become a path for executing unrelated code. The MD5 hash and use of the random module for temporary PIN generation also showed why security-related code needs more care than ordinary application logic.

The mutable default argument and broad exception handler were useful examples of how a program can behave unpredictably even when it appears to work for a simple demonstration. Correcting these issues made the functions more predictable and easier to test. At the same time, static analysis findings still need to be reviewed by a developer. A tool can flag a risky pattern, but the developer must understand the program and choose a fix that preserves the intended behaviour.

8. Conclusion

The revised bank account program addresses the identified issues in input validation, expression evaluation, PIN handling, file operations, naming and module execution. The revised file compiles and passed all 16 focused functional checks. The remaining submission work is to run Pylint, Bandit and Flake8 locally, attach their before-and-after reports, confirm the required Pylint score and create the three separate commits requested by the handout.

Reference material: Assigned Lab 5 - Static Code Analysis Student Handout and the bank account script in the repository listed on the cover page.